Adreno as a native backend in Modular’s compiler

```
mojo build --target-accelerator adreno840 kernel.mojo  →  one Vulkan compute shader (.spv) per kernel
```

Piece	What it does	In the fork
Target	adreno840: Vulkan SPIR-V, 64-lane subgroups, cooperative matrices for the matrix units	SPIRVTraits.cpp
GPU intrinsics	thread ids, barriers, shuffles, warp reductions and math become SPIR-V builtins	SPIRVLowering.cpp
Vulkan kernel ABI	pointers become storage buffers, numbers a params buffer; structured control flow; matrix types	VulkanKernelABI.cpp
SPIR-V writer	LLVM’s SPIR-V backend inside mojo, patched for matrices in Vulkan shaders	SPIRVBackend.cpp, 2 LLVM patches
MAX	CoopMatrix type; matmul picks a matrix-unit kernel on adreno840; graphs compile for vulkan:adreno840	mma_spirv.mojo, matmul/gpu/adreno

Built on Modular’s open-source Mojo compiler · runs on Qualcomm’s stock Vulkan driver

A Mojo kernel, and the SPIR-V it becomes

Subgroup: a lane shuffle and a warp reduction (64 lanes)

```
ss += shuffle_xor(ss, UInt32(8))
var s = warp_sum(x)
```

```
%lane = OpLoad %u32 %SubgroupLocalInvocationId
%src = OpBitwiseXor %u32 %lane %8
%t = OpGroupNonUniformShuffle %f32 %sg %ss %src
%s = OpGroupNonUniformFAdd %f32 %sg Reduce %x
```

Cooperative matrix: one 64 × 64 fp32 tile, from A (64 × 8) and B (8 × 64) in shared memory

```
var acc = Acc.zero()
var a = A.load(sa, 16)
var b = B.load[row_major=False](sb, 16)
acc = a.multiply_add(b, acc)
acc.store(out, ldy)
```

```
%C = OpTypeCooperativeMatrixKHR %f32 %sg 64 64 %Acc
%z = OpConstantComposite %C %f32_0
%a = OpCooperativeMatrixLoadKHR %A %sa %RowMajor %16
%b = OpCooperativeMatrixLoadKHR %B %sb %ColMajor %16
%c = OpCooperativeMatrixMulAddKHR %C %a %b %z
OpCooperativeMatrixStoreKHR %out %c %RowMajor %ldy
```

A, B and Acc are CoopMatrix[DType.float32, ...] with MATRIX_A, MATRIX_B and ACCUMULATOR. Every pointer argument becomes storage buffer 0, 1, 2...; every number a slot of %params.

spirv-dis of normalize_qk, gemv and p_gemm_t2 from mojo-adreno’s builds · ids renamed for reading · %sg = subgroup scope

Mojo kernels, run by a small C program

Write in Mojo
plain GPU kernels, as for any GPU

Compile
one .spv per kernel, checked by spirv-val

Export weights
on the Mac, packed for the kernels, with reference logits

Run with C
on the phone, straight on Vulkan

```
load_kernels(dir);     // .spv → pipelines
record(cmd, plan, n); // every dispatch, once
for (p = 0; p < n_tokens; ++p) {
  *token = tok; *pos = p; // params buffer
  submit_wait(queue, cmd);
  tok = argmax(logits);   // on the CPU
}
```

Model	Mojo kernels	C runner	Dispatches a token
Llama 3.2 1B	9, and 8 for prefill	563 lines	150
Bonsai 2 2B	21 (the port's), 20 for prefill	747 lines	721
Qwen3-8B	7 new, 3 of Bonsai's	414 lines	615

> The C side knows only buffers and dispatches: a kernel's pointers are its bindings in order, its numbers slots of one params buffer.

llama_vk.c, bonsai_vk.c, qwen3_vk.c · built with the Android NDK · Vulkan loaded from the phone's libvulkan.so

Prefill and decode on the Adreno 840

Model	On the GPU	Decode rate	Prefill rate, batched	Prefill rate, one token at a time	Checked
Llama 3.2 1B, fp16	2.47 GB	**23 tok/s** · 43 ms	**684 tok/s** · 0.63 s, 28×	24 tok/s · 17.7 s	PyTorch 48/48
Llama 3.2 1B, 4-bit	1.18 GB	**75 tok/s** · 13 ms	**695 tok/s** · 0.62 s	not measured	PyTorch 48/48
Bonsai 2 2B, ternary	0.48 GB	**80 tok/s** · 12.4 ms	**474 tok/s** · 0.87 s, 6×	80 tok/s · 5.2 s	MLX 16/16
Qwen3-8B, ternary	2.19 GB	**25 tok/s** · 40 ms	not built yet	27 tok/s · 38 ms each	MLX 16/16

Decode: memory bandwidth

One token is a matrix-vector product: its time is the bytes of weights read. Fewer bits per weight took Llama from 43 to 13 ms.

Prefill: the matrix units

All prompt tokens at once reuse every weight: fp32 cooperative-matrix products at about 1.8 TFLOP/s, 6 to 28 times faster.

Prompts: Llama 431 tokens, Bonsai 412, Qwen3 19–23 · Llama and Bonsai decode: Galaxy S26 Ultra · the rest: SM8850 reference phone

Next: MAX itself on the Adreno

26/26 MAX test graphs compile for vulkan:adreno840

MAX’s matmul runs on the matrix units: 3/3 pass

MAX already drives an Adreno through a remote device

Advantages

- One model definition: the same MAX graph as on NVIDIA and AMD, no hand-written runner per model
- The graph compiler fuses and plans memory: far fewer than today’s 615 to 721 dispatches a token
- Pipelines bring the tokenizer, sampling, KV cache and serving
- Compile on the Mac, ship a compiled model to the phone

Challenges

- No Android build of MAX’s runtime, and no on-phone Vulkan device plugin yet
- Kernels must fit Adreno: 64-row tiles, fp16-only sums, 32 KB shared memory
- Ternary and 4-bit formats are custom kernels today
- Driver builds differ: every compiled graph needs checking on the device
- Phone memory is shared and small: weights mapped once, compiling off the device

MAX’s device plugin interface: about 60 C functions · remote device: MAX on the Mac, Vulkan on the phone